Instructivo: MQTT con ESP32, Mosquitto y MQTT Explorer

---

1. ¿Qué es MQTT?

MQTT (Message Queuing Telemetry Transport) es un protocolo de comunicación ligero diseñado para entornos con recursos limitados y conexiones de red poco confiables, lo que lo hace ideal para IoT (Internet of Things).

Utiliza un modelo **publicador/suscriptor (pub/sub)**:

- Un **broker** (servidor central) gestiona todos los mensajes
- Los **dispositivos publican** mensajes en un "topic" (tema)
- Otros dispositivos **suscriben** a esos topics para recibir los datos

Ventajas de MQTT:

- Muy liviano → consume poco ancho de banda y batería
- Funciona bien con conexiones inestables
- Escalable → un broker puede manejar miles de dispositivos
- Simple de implementar

---

2. ¿Qué es Mosquitto?

Mosquitto es un broker MQTT de código abierto desarrollado por Eclipse Foundation. Es un programa que se instala y corre en un servidor (o PC) y actúa como el intermediario central de toda la comunicación MQTT.

- Es **puramente de línea de comandos**, corre en background sin interfaz gráfica
- Puerto estándar: **1883** (sin cifrado) / **8883** (con TLS/SSL)

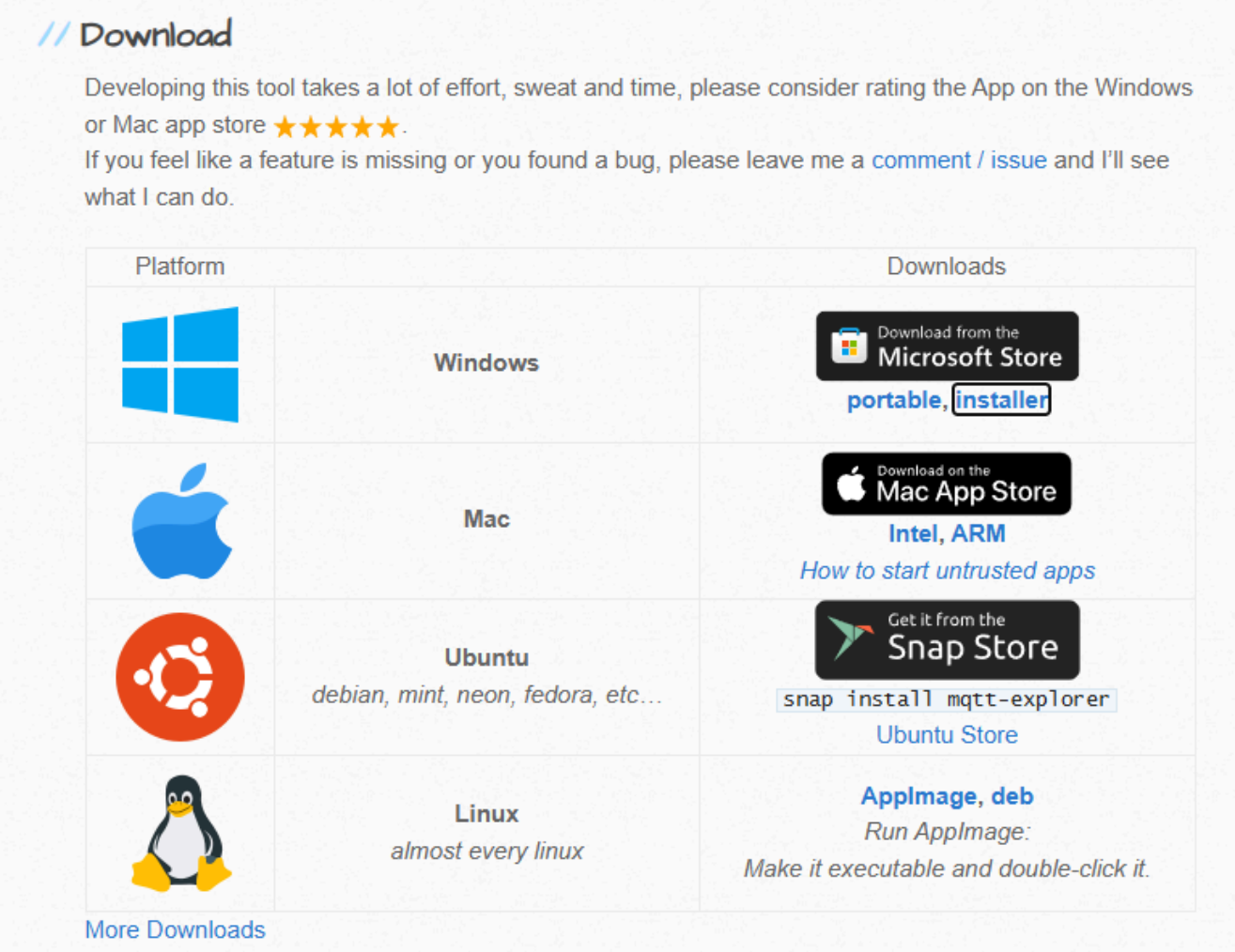
¿Qué es MQTT Explorer?

MQTT Explorer es una aplicación de escritorio con interfaz gráfica que se conecta al broker Mosquitto y permite visualizar todos los topics y mensajes en tiempo real. Es ideal para monitorear y depurar proyectos IoT.

---

3. Instalacion de Mosquitto y MQTT Explorer

- Link de Mosquitto: https://mosquitto.org/download/
- Link de MQTT Explorer: https://mqtt-explorer.com/



---

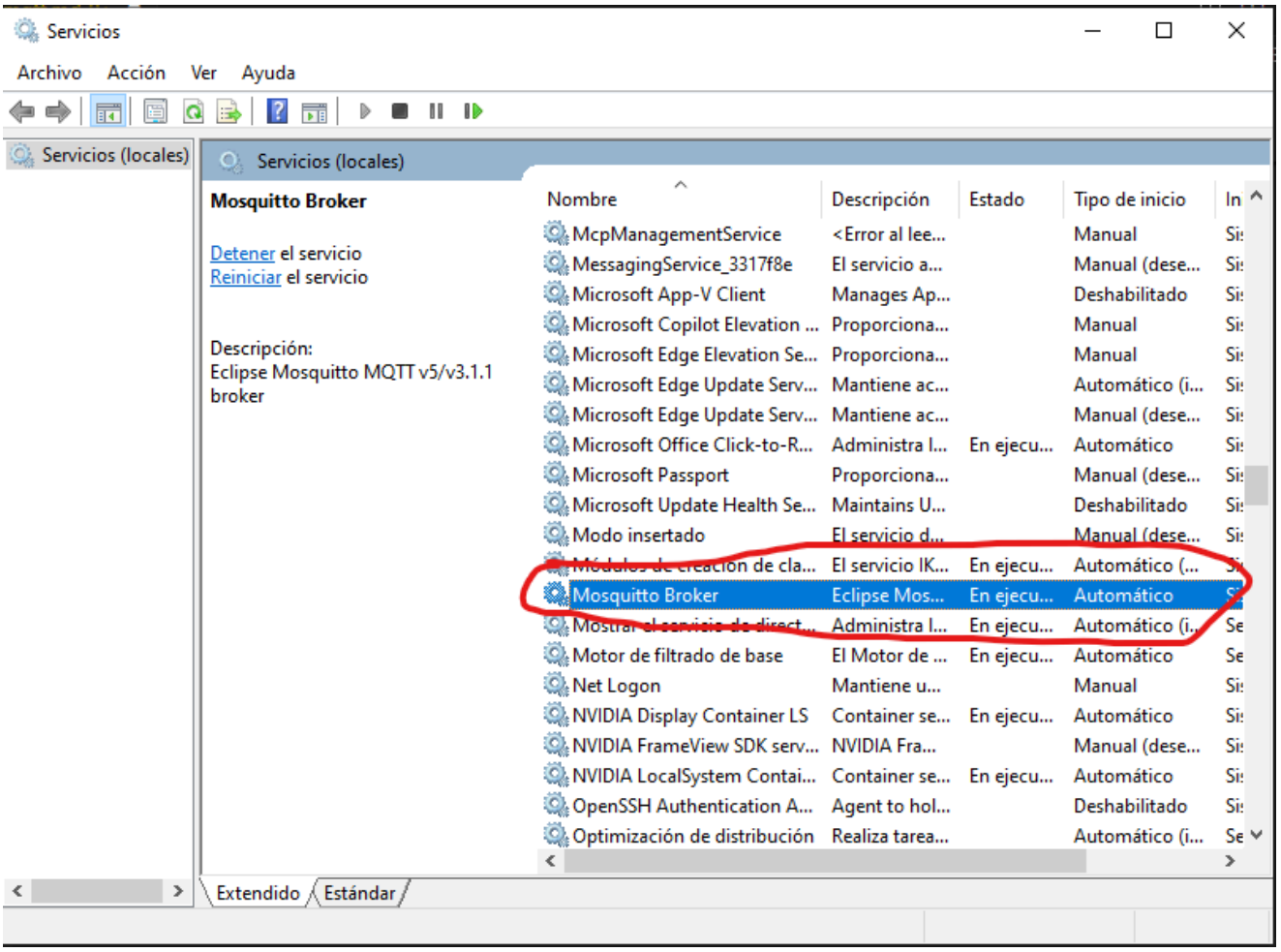
4. Verificar que Mosquitto esté corriendo

Abrir el Administrador de Servicios de Windows:

```
Win + R → escribir: services.msc → Enter
```

Buscar **Mosquitto** en la lista y verificar que el estado diga **"En ejecución"**.

Si no está corriendo: click derecho → **Iniciar**.

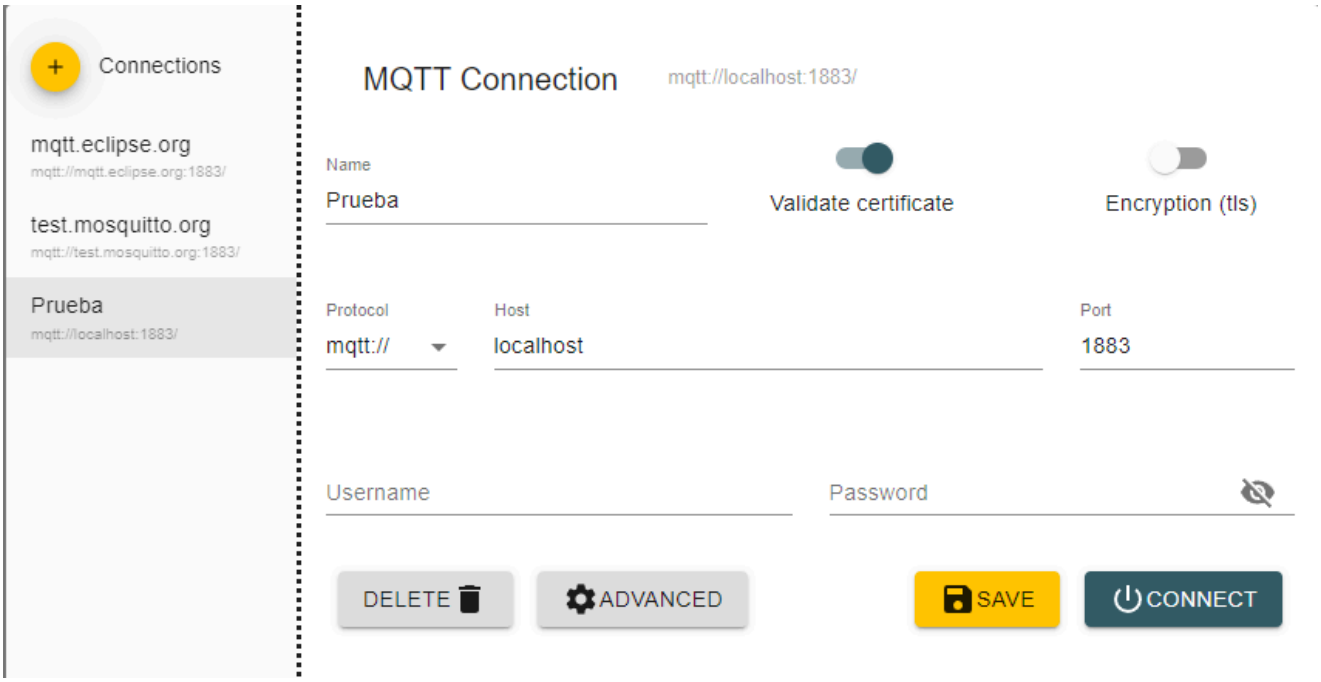


5. Conectar MQTT Explorer al broker

Al abrir MQTT Explorer, completar los siguientes campos:

Campo	Valor
Name	Mosquitto Local (o cualquier nombre)
Protocol	`mqtt://`
Host	`localhost` o `127.0.0.1`
Port	`1883`
Username	*(vacío)*
Password	*(vacío)*

Luego hacer click en **CONNECT**.



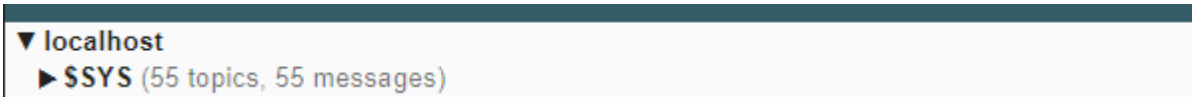
6. ¿Qué es $SYS?

Una vez conectado, aparece automáticamente el topic **$SYS**. Es un topic especial que Mosquitto publica con información sobre su propio estado. Es el "monitor de salud" del broker.

Al expandirlo se puede ver:

```
$SYS
    broker
        ├── clients/connected   → clientes conectados actualmente
        ├── messages/received   → total de mensajes recibidos
        ├── messages/sent       → total de mensajes enviados
        ├── uptime              → tiempo que lleva corriendo el broker
        └── version             → versión de Mosquitto instalada
```

Si se ve `$SYS` en MQTT Explorer significa que **Mosquitto está funcionando correctamente**.



7. Asegurarse de estar en la misma red WiFi

Requisito fundamental: El ESP32 y la PC tienen que estar conectados a la **misma red WiFi**.

Verificar que en el código del ESP32 el nombre y contraseña de la red sean correctos:

```
const char* ssid     = "NombreDeTuWifi";  // Sensible a mayúsculas
const char* password = "TuContraseña";
```

Dispositivo	Red WiFi
PC	Conectada al mismo WiFi
ESP32 #1	Mismo nombre y contraseña
ESP32 #2	Mismo nombre y contraseña

> ⚠️ Si la PC está conectada por cable Ethernet y el ESP32 por WiFi, pueden estar en subredes distintas. Conectar la PC también por WiFi para evitar este problema.

---

8. Verificar la IP de la PC

Abrir CMD y escribir:

```
ipconfig
```

Buscar la sección **Adaptador de LAN inalámbrica Wi-Fi** y anotar el valor de **Dirección IPv4**:

```
Adaptador de LAN inalámbrica Wi-Fi:
   Dirección IPv4. . . . . . . . . . : 192.168.X.X  ← esta es la IP
```

> ⚠️ Usar siempre la IP del adaptador **Wi-Fi**, no de Ethernet, VPN ni Bluetooth.

Esta IP es la que hay que poner en el código del ESP32:

```
const char* mqtt_server = "192.168.X.X"; // IP real de la PC
```

---

9. Verificar que Mosquitto escucha conexiones externas

Abrir CMD y ejecutar:

```
netstat -an | findstr 1883
```

Resultado correcto ✅

```
TCP    0.0.0.0:1883    0.0.0.0:0    LISTENING
```

`0.0.0.0` significa que Mosquitto acepta conexiones desde cualquier dispositivo de la red.

Resultado incorrecto ❌

```
TCP    127.0.0.1:1883    0.0.0.0:0    LISTENING
```

`127.0.0.1` significa que Mosquitto solo acepta conexiones locales (desde la misma PC). En este caso se debe modificar el archivo de configuración (ver paso 10).

---

10. Modificar el archivo de configuración de Mosquitto

Si el netstat muestra `127.0.0.1` en lugar de `0.0.0.0`, seguir estos pasos:

1. Abrir el Bloc de notas como Administrador:

```
Click derecho en Bloc de notas → Ejecutar como administrador
```

2. Abrir el archivo:

```
C:\Program Files\mosquitto\mosquitto.conf
```

3. Agregar las siguientes líneas al **final del archivo** (sin el símbolo `#`):

```
listener 1883
allow_anonymous true
```

> ⚠️ Las líneas que empiezan con `#` son comentarios y Mosquitto las ignora. Las líneas nuevas deben agregarse **sin** `#`.

4. Guardar el archivo.

5. Reiniciar el servicio Mosquitto:

```
Win + R → services.msc → buscar Mosquitto → click derecho → Reiniciar
```

6. Verificar nuevamente con netstat que ahora muestre `0.0.0.0:1883`.

---

11. Permitir el puerto 1883 en el Firewall de Windows

Abrir CMD **como administrador** y ejecutar:

```
netsh advfirewall firewall add rule name="Mosquitto" dir=in action=allow
protocol=TCP localport=1883
```

Si el comando responde **"Aceptar"** o **"Ok"**, la regla fue creada correctamente y Windows permitirá las conexiones entrantes al broker Mosquitto desde los ESP32.

---

12. Programar la EPS32 numero uno

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid        = "NOMBRE RED WIFI";
const char* password    = "CONTRASEÑA RED WIFI";
const char* mqtt_server = "192.168.1.114"; // IP de tu PC

WiFiClient espClient;
PubSubClient client(espClient);

void callback(char* topic, byte* message, unsigned int length) {
  String msg = "";
  for (int i = 0; i < length; i++) msg += (char)message[i];
  Serial.println("Daniel dice: " + msg);
}

void setup_wifi() {
  WiFi.begin(ssid, password);
  Serial.print("Conectando WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi conectado!");
}

void reconnect() {
  while (!client.connected()) {
    Serial.println("HOLA");
    if (client.connect("Gaspar")) {
      Serial.println("Gaspar conectado al broker!");
      client.subscribe("daniel/saludo"); // Gaspar escucha a Daniel
    } else {
      delay(2000);
    }
  }
```

```
}

void setup() {
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) reconnect();
  client.loop();

  static unsigned long lastMsg = 0;
  if (millis() - lastMsg > 5000) {
    lastMsg = millis();
    client.publish("gaspar/saludo", "Hola Daniel, soy Gaspar!");
    Serial.println("Gaspar: mensaje enviado!");
  }
}
```

---

13. Programar la EPS32 numero dos

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid        = "ArrigoWifi2.4Ghz";
const char* password    = "Lunamiperra10";
const char* mqtt_server = "192.168.1.114"; // IP de tu PC

WiFiClient espClient;
PubSubClient client(espClient);

void callback(char* topic, byte* message, unsigned int length) {
  String msg = "";
  for (int i = 0; i < length; i++) msg += (char)message[i];
  Serial.println("Gaspar dice: " + msg);
}

void setup_wifi() {
  WiFi.begin(ssid, password);
  Serial.print("Conectando WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
```

```
  Serial.println("\nWiFi conectado!");
}

void reconnect() {
  while (!client.connected()) {
    if (client.connect("Daniel")) {
      Serial.println("Daniel conectado al broker!");
      client.subscribe("gaspar/saludo"); // Daniel escucha a Gaspar
    } else {
      delay(2000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) reconnect();
  client.loop();

  static unsigned long lastMsg = 0;
  if (millis() - lastMsg > 5000) {
    lastMsg = millis();
    client.publish("daniel/saludo", "Hola Gaspar, soy Daniel!");
    Serial.println("Daniel: mensaje enviado!");
  }
}
```

Verificación final

Una vez completados todos los pasos, en el **Monitor Serie** del ESP32 (115200 baudios) se debería ver:

```
Conectando WiFi.........
WiFi conectado! IP: 192.168.X.X
Conectando MQTT...
Conectado!
Gaspar: mensaje enviado!
```

Y en **MQTT Explorer** aparecerán los topics de los ESP32 junto al $SYS:

```
📂 $SYS
📂 gaspar
    └── saludo → "Hola Daniel, soy Gaspar!"
📂 daniel
    └── saludo → "Hola Gaspar, soy Daniel!"
```